Introduction to MiniZinc

Jacopo Mauro

Department of Mathematics & Computer Science
University of Southern Denmark

[*Based on slides by Marco Chiarandini*]

MiniZinc

- language for specifying: constrained optimization and decision problems over integers and real numbers.
- MiniZinc compiler transforms model file + data file into FlatZinc model
- target the specific solvers, such as Constraint Programming (CP), Mixed Integer Linear Programming (MIP) or Boolean Satisfiability (SAT) solvers.
- Website: `www.minizinc.org`
- Tutorial: `https://docs.minizinc.dev/en/stable/part_2_tutorial.html`

A First Model

```
% Colouring Australia using nc colours
int: nc = 3;
var 1..nc: wa; var 1..nc: nt; var 1..nc: sa; var 1..nc: q;
var 1..nc: nsw; var 1..nc: v; var 1..nc: t;

constraint wa != nt;
constraint wa != sa;
constraint nt != sa;
constraint nt != q;
constraint sa != q;
constraint sa != nsw;
constraint sa != v;
constraint q != nsw;
constraint nsw != v;

solve satisfy;

output ["wa=\(wa)\t nt=\(nt)\t sa=\(sa)\n",
"q=\(q)\t nsw=\(nsw)\t v=\(v)\n",
"t=", show(t), "\n"];
```

Parameters and Variables

- comments % or `/* */`
- types are integers (`int`), floating point numbers (`float`), Booleans (`bool`) and strings (`string`). Further: enum; sets and (multi-dimensional) arrays.
- decision variables are variables in the sense of mathematical or logical variables not in the sense of programming language variables.
- the domain can be given as part of the variable declaration and the type is inferred from there
- decision variables can be Booleans, integers, floating point numbers, or sets. Moreover, arrays of decision variables
- identifiers are made of characters or _ and must start with a char.

Parameters:

```
int : <var-name>
<l>..<u> : <var-name>
```

Decision variables:

```
var int : <var-name>
var <l>..<u> : <var-name>
```

Example of variable declration:

```
var int: i; constraint i >=0 /\ i <= 4;
var 0..4: i;
var {0,1,2,3,4}: i;
```

Constraints

Relational operators defined for built-in types:

equal	= or ==
not equal	!=
strictly less than	<
strictly greater than	>
less than or equal to	<=
greater than or equal to	>=

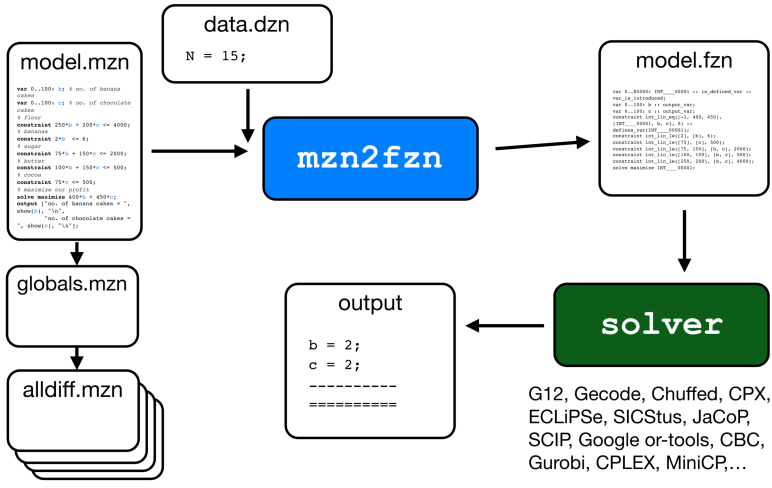
Output

- between quotes for strings and enclosed by a `show` call if MiniZinc elements (or interpolation `"\(e)"`).
- string concatenation via operator ++
- formatted output: `show_int(n,X)` and `show_float(n,d,X)`

Basic Structure

```
include <filename>;

<type inst expr>: <variable> [ = ] <expression>;

<variable> = <expression>;

constraint <Boolean expression>;

solve satisfy;
solve maximize <arithmetic expression>;
solve minimize <arithmetic expression>;

output [ <string expression>, ..., <string expression> ];
```

Workflow


data.dzn
N = 15;
model.mzn
var 0..100: b; % no. of banana cakes
var 0..100: c; % no. of chocolate cakes
% flour
constraint 250*b + 200*c <= 4000;
% bananas
constraint 2*b <= 6;
% sugar
constraint 75*b + 150*c <= 2000;
% butter
constraint 100*b + 150*c <= 500;
% cocoa
constraint 75*c <= 500;
% maximise our profit
solve maximize 400*b + 450*c;
output ["no. of banana cakes = ", show(b), "\n",
"no. of chocolate cakes = ", show(c), "\n"];
mzn2fzn
model.fzn
var 0..85000: INT____00001 :: is_defined_var :: var_is_introduced;
var 0..100: b :: output_var;
var 0..100: c :: output_var;
constraint int_lin_eq([-1, 400, 450], [INT____83001, b, c], 0) :: defines_var(INT____00001);
constraint int_lin_le([2], [b], 6);
constraint int_lin_le([75], [c], 500);
constraint int_lin_le([75, 150], [b, c], 2000);
constraint int_lin_le([100, 150], [b, c], 500);
constraint int_lin_le([250, 200], [b, c], 4000);
solve maximize INT____00001;
globals.mzn
alldiff.mzn
solver
output
b = 2;
c = 2;

==========
G12, Gecode, Chuffed, CPX, ECLiPSe, SICStus, JaCoP, SCIP, Google or-tools, CBC, Gurobi, CPLEX, MiniCP,…

Another Example: Production Planning

```
% Baking cakes for the school party

% Number of cakes
var 0..100: b; % no. of banana cakes
var 0..100: c; % no. of chocolate cakes

% Parameters
int: flour;
int: banana;
int: sugar;
int: butter;
int: cocoa;

% Constraints
constraint 250*b + 200*c <= flour;
constraint 2*b <= banana;
constraint 75*b + 150*c <= sugar;
constraint 100*b + 150*c <= butter;
constraint 75*c <= cocoa;

% Maximize our profit
solve maximize 400*b + 450*c;

% Output
output ["no. of banana cakes = \(b)\n",
        "no. of chocolate cakes = \(c)\n"];
```

Arithmetics:

Addition	+	Integer division	div
Subtraction	-	Integer modulus	mod
Multiplication	*	Absolute value	abs
Power function	pow	Float division	/

```
% dzn file
flour = 4000;
banana = 6;
sugar = 2000;
butter = 500;
cocoa = 500;
```

Arrays

one- and multi-dimensional arrays:

- declared as `array[index_set] of type`
- can contain anything except other arrays
- index set must be range of integers

Examples:

- array literals: `[v1,v2,v3]`
- concatenation: `[1,2,3] ++ [4,5,6]`
- 2d literals: `[|1,2,3|1,2,3|]`
- index set coercion: array2d, array3d, etc.

N Queens

```
int: n;
% map each queen to its column
array[1..n] of var 1..n: q;

% no two queens in the same column
constraint forall (i in 1..n, j in 1..n where i!=j) (
  q[i] != q[j]   );

% no two queens in the same diagonal
constraint forall (i in 1..n, j in 1..n where i!=j) (
  q[i] + i != q[j] + j);
constraint forall (i in 1..n, j in 1..n where i!=j) (
  q[i] - i != q[j] - j
);

solve satisfy;

output [show(q)];
```

Sets

For parameters: sets of integers, enums, floats or Booleans.
For variables: sets of integers or enums.

```
set of <type-inst> : <var-name> ;
{ <expr-1>, ..., <expr-n> } % set
 <expr-1> .. <expr-2> % range
```

Set operations

element membership	`in`
(non-strict) subset relationship	`subset`
(non-strict) superset relationship	`superset`
union	`union`
inter-section	`intersect`
set difference	`diff`
symmetric set difference	`symdiff`
number of elements in the set	`card`

Enumerations

Enumerated types `enums`

```
enum <var-name> ; % declaration as an unknown set of elements
enum <var-name> = { <var-name-1>, ..., <var-name-n> } ; % definition via assignment
```

```
enum Color;
Color = { red, yellow, blue };
```

Elements of an enumerated type of n elements internally are represented as integers $1 \ldots n$:

- they can be compared
- they are ordered, by the order they appear in the enumerated type definition,
- they can be iterated over,
- they can appear as indices of arrays,

They can be used for indexing

```
array[Products] of int: profit;
```

Built-in functions

forall takes a one-dimensional array and aggregates the elements.
The array is made of Boolean expressions (that is, constraints) and **forall** returns a single Boolean expression which is the logical conjunction

```
array of 1..3: a;
forall( [a[i] != a[j] | i,j in 1..3 where i < j]);
forall(i,j in 1..3 where i < j) ([a[i] != a[j]); % equivalent to the above one
```

The expressions mean:

```
a[1] != a[2] /\ a[1] != a[3] /\ a[2] != a[3]
```

Aggregation Functions

Aggregation functions:

- for arithmetic arrays are: **sum**, **product**, **min**, **max**
  When applied to an empty array, **sum** returns 0, **product** returns 1, **min** and **max** give a run-time error.

- for arrays containing Boolean expressions are: **forall** (logical conjunction, all hold), **exists** (logical disjunction, at least one holds), **xorall** (ensures that an odd number hold), **iffall** (an even number of holds).

Better looking version of the Australia Problem

```
% Define the regions of Australia
enum Region = {WA, NT, SA, Q, NSW, V, T};

% Define the three colors available
enum Color = {Red, Green, Blue};

% Decision variables: Each region is assigned a color
array[Region] of var Color: color;

% Explicit adjacency constraints between regions
constraint color[WA]  != color[NT];
constraint color[WA]  != color[SA];
constraint color[NT]  != color[SA];
constraint color[NT]  != color[Q];
constraint color[SA]  != color[Q];
constraint color[SA]  != color[NSW];
constraint color[SA]  != color[V];
constraint color[Q] != color[NSW];
constraint color[NSW] != color[V];

solve satisfy;

output ["Region " ++ show(r) ++ ": " ++ show(color[r]) ++ "\n" | r in Region];
```

Global Constraints

```
var 1..9: T;
var 0..9: H;
var 1..9: I;
var 0..9: S;
var 1..9: E;
var 0..9: A;
var 0..9: Y;

include "globals.mzn";

constraint alldifferent([T,H,I,S,E,A,Y]);

constraint 1000*T + 100*H + 10*I + S
  + 10*I + S
  = 1000*E + 100*A + 10*S + Y;

  solve satisfy;
```

```
  T H I S
+     I S
= E A S Y
```

List of global constraints at https://docs.minizinc.dev/en/2.2.3/lib-globals.html, included by using **include** "globals.mzn".